

2026.03.03.화

🕒 생성일	@2026년 3월 3일 오전 9:06
🏷 태그	AWS Kubernetes

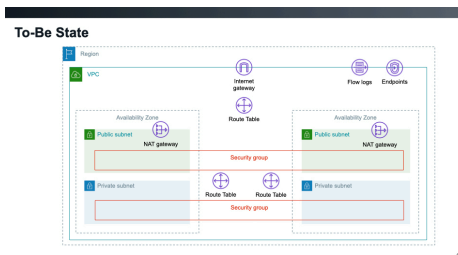
목차

1. ECR, ECS, EKS, Github Actions 복습
2. AWS CloudFormation 실습

▼ 1) 복습

- ECR
 - Docker 설치
 - AWS cli 설치
 - AWS 콘솔에서 ECR repo 생성
 - Docker에 있는 이미지 푸시
- ECS
 - 클러스터 생성
 - 태스크 정의
 - 서비스 생성
- EKS
 - kubernetes 설치 (k8s, k3s, minikube 중 택)
 - hostname 변경
 - 마스터 노드, 워커 노드 등록
 - EKS pod 생성
 - 워커 노드에서 ECR 이미지 pull
 - pod 실행
- Github Actions
 - github repo 생성
 - git init, remote
 - 환경 변수 등록
 - .github/workflow 안에 .yaml 파일 생성
 - git push

▼ 2) AWS CloudFormation 실습



- 기본적인 아키텍처 yaml 파일로 구현
- 추가로 public subnet에 bastion ec2 서버까지 구현

```

# VPC

AWSTemplateFormatVersion: 2010-09-09

Description: Custom VPC

Metadata:
  AWS::CloudFormation::Interface:
    ParameterGroups:
      - Label:
          default: Custom VPC
        Parameters:
          - ParamVpcCidrBlock
      - Label:
          default: Subnets
        Parameters:
          - ParamPublicSubnetACidrBlock
          - ParamPrivateSubnetACidrBlock
          - ParamPublicSubnetCCidrBlock
          - ParamPrivateSubnetCCidrBlock
      - Label:
          default: Availability Zones
        Parameters:
          - ParamAZone
          - ParamCZone
      - Label:
          default: Bastion Host # 콘솔에서 Bastion 관련 파라미터를 그룹으로 표시
        Parameters:
          - ParamKeyPair
          - ParamBastionInstanceType
          - ParamBastionAmiId
    # =====

Parameters:
  ParamVpcCidrBlock: # User-given Logical ID
    Type: String
    Description: VPC CIDR block
    Default: '10.0.0.0/16'

  ParamPublicSubnetACidrBlock:
    Type: String
    Description: Public Subnet A-zone CIDR block
    Default: '10.0.1.0/24'
  ParamPrivateSubnetACidrBlock:
    Type: String
    Description: Private Subnet A-zone CIDR block
    Default: '10.0.2.0/24'
  ParamPublicSubnetCCidrBlock:
    Type: String
    Description: Public Subnet C-zone CIDR block
    Default: '10.0.3.0/24'
  ParamPrivateSubnetCCidrBlock:
    Type: String
    Description: Private Subnet C-zone CIDR block
    Default: '10.0.4.0/24'

  ParamAZone:
    Type: 'AWS::EC2::AvailabilityZone::Name'

```

```

    Description: Availability Zone A
    Default: 'ap-northeast-2a'
ParamCZone:
    Type: 'AWS::EC2::AvailabilityZone::Name'
    Description: Availability Zone C
    Default: 'ap-northeast-2c'

# =====
# SSH 접속을 위한 키 페어 (AWS 콘솔에서 미리 생성해야 함)
ParamKeyPair:
    Type: 'AWS::EC2::KeyPair::KeyName' # 계정에 존재하는 키 페어 목록에서 선택
    Description: Key pair for Bastion EC2 instance
# Bastion 인스턴스 타입 (프리 티어: t2.micro)
ParamBastionInstanceType:
    Type: String
    Description: Bastion EC2 instance type
    Default: 't3.micro'
    AllowedValues: # 허용되는 인스턴스 타입을 제한
        - t3.micro
        - t3.small
        - t3.medium
# SSM Parameter Store에서 최신 Amazon Linux 2023 AMI ID를 자동으로 가져옴
# 배포 시점마다 항상 최신 AMI를 사용하므로 하드코딩할 필요 없음
ParamBastionAmiId:
    Type: 'AWS::SSM::Parameter::Value<AWS::EC2::Image::Id>'
    Description: Bastion EC2 AMI ID (Amazon Linux 2023 latest)
    Default: '/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64'
# =====

Resources:
    # VPC

    MyVPC: # User-given Logical ID
        Type: 'AWS::EC2::VPC'
        Properties:
            CidrBlock: !Ref ParamVpcCidrBlock # Short form (YAML)

    # Subnet (4)

    MyPublicSubnetA:
        Type: 'AWS::EC2::Subnet'
        Properties:
            AvailabilityZone: !Ref ParamAZone
            CidrBlock: !Ref ParamPublicSubnetACidrBlock
            MapPublicIpOnLaunch: true
            VpcId: !Ref MyVPC
    MyPrivateSubnetA:
        Type: 'AWS::EC2::Subnet'
        Properties:
            AvailabilityZone: !Ref ParamAZone
            CidrBlock: !Ref ParamPrivateSubnetACidrBlock
            MapPublicIpOnLaunch: true
            VpcId: !Ref MyVPC
    MyPublicSubnetC:
        Type: 'AWS::EC2::Subnet'
        Properties:
            AvailabilityZone: !Ref ParamCZone
            CidrBlock: !Ref ParamPublicSubnetCCidrBlock
            MapPublicIpOnLaunch: true
            VpcId: !Ref MyVPC

```

```

MyPrivateSubnetC:
  Type: 'AWS::EC2::Subnet'
  Properties:
    AvailabilityZone: !Ref ParamCZone
    CidrBlock: !Ref ParamPrivateSubnetCCidrBlock
    MapPublicIpOnLaunch: true
    VpcId: !Ref MyVPC

# Internet Gateway
# VPC Gateway Attachment

MyInternetGateway:
  Type: 'AWS::EC2::InternetGateway'
MyIGWAttachment:
  Type: 'AWS::EC2::VPCGatewayAttachment'
  Properties:
    InternetGatewayId: !Ref MyInternetGateway
    VpcId: !Ref MyVPC

# Elastic IP (2)
# NAT Gateway (2)

MyEipA:
  Type: 'AWS::EC2::EIP'
  DependsOn: MyIGWAttachment # required
  Properties:
    Domain: vpc
MyEipC:
  Type: 'AWS::EC2::EIP'
  DependsOn: MyIGWAttachment # required
  Properties:
    Domain: vpc
MyNatGatewayA:
  Type: 'AWS::EC2::NatGateway'
  Properties:
    AllocationId:
      Fn::GetAtt:
        - MyEipA
        - AllocationId
    SubnetId: !Ref MyPublicSubnetA
MyNatGatewayC:
  Type: 'AWS::EC2::NatGateway'
  Properties:
    AllocationId:
      Fn::GetAtt:
        - MyEipC
        - AllocationId
    SubnetId: !Ref MyPublicSubnetC

# RouteTable (3)
# Route (3)

MyPublicRouteTable:
  Type: 'AWS::EC2::RouteTable'
  Properties:
    VpcId: !Ref MyVPC
MyPublicRoute:
  Type: 'AWS::EC2::Route'
  Properties:
    DestinationCidrBlock: 0.0.0.0/0

```

```

    GatewayId: !Ref MyInternetGateway
    RouteTableId: !Ref MyPublicRouteTable

MyPrivateRouteTableA:
  Type: 'AWS::EC2::RouteTable'
  Properties:
    VpcId: !Ref MyVPC
MyPrivateRouteA:
  Type: 'AWS::EC2::Route'
  Properties:
    DestinationCidrBlock: 0.0.0.0/0
    NatGatewayId: !Ref MyNatGatewayA
    RouteTableId: !Ref MyPrivateRouteTableA

MyPrivateRouteTableC:
  Type: 'AWS::EC2::RouteTable'
  Properties:
    VpcId: !Ref MyVPC
MyPrivateRouteC:
  Type: 'AWS::EC2::Route'
  Properties:
    DestinationCidrBlock: 0.0.0.0/0
    NatGatewayId: !Ref MyNatGatewayC
    RouteTableId: !Ref MyPrivateRouteTableC

# SubnetRouteTableAssociation (4)

MyPublicSubnetARouteTableAssociation:
  Type: 'AWS::EC2::SubnetRouteTableAssociation'
  Properties:
    RouteTableId: !Ref MyPublicRouteTable
    SubnetId: !Ref MyPublicSubnetA
MyPrivateSubnetARouteTableAssociation:
  Type: 'AWS::EC2::SubnetRouteTableAssociation'
  Properties:
    RouteTableId: !Ref MyPrivateRouteTableA
    SubnetId: !Ref MyPrivateSubnetA
MyPublicSubnetCRouteTableAssociation:
  Type: 'AWS::EC2::SubnetRouteTableAssociation'
  Properties:
    RouteTableId: !Ref MyPublicRouteTable
    SubnetId: !Ref MyPublicSubnetC
MyPrivateSubnetCRouteTableAssociation:
  Type: 'AWS::EC2::SubnetRouteTableAssociation'
  Properties:
    RouteTableId: !Ref MyPrivateRouteTableC
    SubnetId: !Ref MyPrivateSubnetC

# VPC Flow Logs

MyFlowLogLogGroup:
  Type: 'AWS::Logs::LogGroup'

MyFlowLogRole:
  Type: 'AWS::IAM::Role'
  Properties:
    AssumeRolePolicyDocument:
      Version: '2012-10-17'
      Statement:
        - Effect: 'Allow'

```

```

        Principal:
          Service: 'vpc-flow-logs.amazonaws.com'
          Action: 'sts:AssumeRole'
    Policies:
      - PolicyName: 'MyFlowLogRolePolicy'
        PolicyDocument:
          Version: '2012-10-17'
          Statement:
            - Effect: 'Allow'
              Action: 'logs:*'
              Resource: '*'

MyFlowLog:
  Type: 'AWS::EC2::FlowLog'
  Properties:
    DeliverLogsPermissionArn:
      Fn::GetAtt:
        - MyFlowLogRole
        - Arn
    LogGroupName: !Ref MyFlowLogLogGroup
    ResourceId: !Ref MyVPC
    ResourceType: VPC
    TrafficType: ALL

# VPC Endpoint

MyS3VpcEndpoint:
  Type: 'AWS::EC2::VPCEndpoint'
  Properties:
    PolicyDocument:
      Version: '2012-10-17'
      Statement:
        - Effect: 'Allow'
          Principal: '*'
          Action: '*'
          Resource: '*'
    RouteTableIds:
      - !Ref MyPublicRouteTable
      - !Ref MyPrivateRouteTableA
      - !Ref MyPrivateRouteTableC
    ServiceName: com.amazonaws.ap-northeast-2.s3
    # ServiceName: !Sub com.amazonaws.${AWS::Region}.s3
    VpcId: !Ref MyVPC

# Security Groups

MyPublicSecurityGroup:
  Type: 'AWS::EC2::SecurityGroup'
  Properties:
    GroupDescription: Allow HTTP (80) and SSH (22) traffic from the Internet
    SecurityGroupIngress:
      - IpProtocol: tcp
        FromPort: '80'
        ToPort: '80'
        CidrIp: 0.0.0.0/0
      - IpProtocol: tcp
        FromPort: '22'
        ToPort: '22'
        CidrIp: 0.0.0.0/0
    VpcId: !Ref MyVPC

```

```
# =====
# Bastion EC2 Instance
# Public Subnet A에 배치하여 외부에서 SSH 접속 가능한 점프 서버 역할
# 이 인스턴스를 경유하여 Private Subnet의 서버에 접속

MyBastionInstance:
  Type: 'AWS::EC2::Instance'
  DependsOn: MyIGWAttachment # IGW가 먼저 연결되어야 Public IP로 통신 가능
  Properties:
    ImageId: !Ref ParamBastionAmiId # Amazon Linux 2023 최신 AMI
    InstanceType: !Ref ParamBastionInstanceType # 기본값: t2.micro
    KeyName: !Ref ParamKeyPair # SSH 접속용 키 페어
    NetworkInterfaces:
      - AssociatePublicIpAddress: true # Public IP 자동 할당 (외부 접속용)
        DeviceIndex: '0' # 기본 네트워크 인터페이스
        SubnetId: !Ref MyPublicSubnetA # Public Subnet A (10.0.1.0/24)에 배치
        GroupSet:
          - !Ref MyPublicSecurityGroup # SSH(22), HTTP(80) 인바운드 허용
    Tags:
      - Key: Name
        Value: Bastion Host
# =====

MyPrivateSecurityGroup:
  Type: 'AWS::EC2::SecurityGroup'
  Properties:
    GroupDescription: Allow HTTP (80) and SSH (22) traffic only from the same VPC
    SecurityGroupIngress:
      - IpProtocol: tcp
        FromPort: '80'
        ToPort: '80'
        CidrIp: !GetAtt MyVPC.CidrBlock
      - IpProtocol: tcp
        FromPort: '22'
        ToPort: '22'
        CidrIp: !GetAtt MyVPC.CidrBlock
    VpcId: !Ref MyVPC
```